

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнил:

Студент 4 курса

Группы ИИ-26

Суховило В. С.

Проверил:

Никитин С. К.

Брест 2026

Цель работы: Реализовать многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.

Ход работы:

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
 - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
 - итоговая суммарная ошибка, точность (ассигасу) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c_0 или c_1) оно ближе.

Вариант 3 ($c_0 = -9$, $c_1 = -8$)

A	B	$A \oplus B$
-9	-9	-9
-9	-8	-8
-8	-9	-8
-8	-8	-9

Код программы:

```
import numpy as np
```

```
#=====
# ЛАБОРАТОРНАЯ РАБОТА №1
# Вариант 3
# c0 = -9, c1 = -8
#=====
```

```
c0, c1 = -9, -8
```

```
# -----
# Таблица истинности XOR для варианта 3
#
# A   B   A XOR B
# -9 -9   -9
# -9 -8   -8
# -8 -9   -8
# -8 -8   -9
# -----
```

```
X = np.array([
    [-9, -9],
    [-9, -8],
    [-8, -9],
    [-8, -8]
], dtype=float)
```

```
Y_raw = np.array([
    [-9],
    [-8],
    [-8],
    [-9]
], dtype=float)
```

```
# =====
# НОРМАЛИЗАЦИЯ И ДЕНОРМАЛИЗАЦИЯ
# =====
```

```
def normalize(x, xmin, xmax):
    """
    Перевод значений из диапазона [xmin, xmax]
    в диапазон [0, 1].
    """
    return (x - xmin) / (xmax - xmin)
```

```
def denormalize(y_norm, xmin, xmax):
    """
    Обратное преобразование из [0, 1]
    в исходный диапазон.
    """
    return xmin + y_norm * (xmax - xmin)
```

```
# -9 -> 0
# -8 -> 1
```

```
Xn = normalize(X, c0, c1)
Y = normalize(Y_raw, c0, c1)
```

```
#=====
# ФУНКЦИЯ АКТИВАЦИИ
#=====
```

```
def sigmoid(S):
    return 1.0 / (1.0 + np.exp(-S))
```

```
def dsigmoid(y):
    """
    Производная сигмоиды.
    На вход передаётся уже вычисленное значение sigmoid.
    """
    return y * (1.0 - y)
```

```
#=====
# МНОГОСЛОЙНЫЙ ПЕРСЕПТРОН 2-2-1
#=====
```

```
class MLP221:

    def __init__(self, lr=0.5, seed=7):

        rng = np.random.RandomState(seed)

        # Веса между входным и скрытым слоем
        self.W1 = rng.uniform(-0.1, 0.1, (2, 2))

        # Пороговые значения скрытого слоя
        self.T1 = rng.uniform(-0.1, 0.1, (1, 2))

        # Веса между скрытым и выходным слоем
        self.W2 = rng.uniform(-0.1, 0.1, (2, 1))

        # Порог выходного слоя
        self.T2 = rng.uniform(-0.1, 0.1, (1, 1))

        # Скорость обучения
        self.lr = lr

    def forward(self, x):
        """
        Прямое распространение сигнала.
        """

        x = x.reshape(1, -1)

        # Скрытый слой
```

```

y1 = sigmoid(x @ self.W1 - self.T1)

# Выходной слой
y2 = sigmoid(y1 @ self.W2 - self.T2)

return y1, y2

def train_step(self, x, e):
    """
    Один шаг обучения методом
    обратного распространения ошибки.
    """

    x = x.reshape(1, -1)
    e = e.reshape(1, -1)

    # Прямой проход
    y1, y2 = self.forward(x)

    # Ошибка выходного слоя
    delta2 = (y2 - e) * dsigmoid(y2)

    # Ошибка скрытого слоя
    delta1 = (delta2 @ self.W2.T) * dsigmoid(y1)

    # Обновление весов выходного слоя
    self.W2 -= self.lr * (y1.T @ delta2)

    # Обновление порога выходного слоя
    self.T2 += self.lr * delta2

    # Обновление весов скрытого слоя
    self.W1 -= self.lr * (x.T @ delta1)

    # Обновление порогов скрытого слоя
    self.T1 += self.lr * delta1

    # Квадратичная ошибка
    error = 0.5 * np.sum((y2 - e) ** 2)

    return error

def predict(self, x):
    """
    Получение результата сети.
    """

    return self.forward(x)[1][0, 0]

```

```
def fit(self, X, Y, Ee=0.01, max_epochs=20000, seed=42):
```

```
    """
```

```
    Обучение сети.
```

```
    Ee - допустимая суммарная ошибка.
```

```
    max_epochs - максимальное число эпох.
```

```
    """
```

```
    rng = np.random.RandomState(seed)
```

```
    for epoch in range(1, max_epochs + 1):
```

```
        # Случайный порядок обучающих примеров
```

```
        idx = rng.permutation(len(X))
```

```
        Es = 0.0
```

```
        # Онлайн-режим:
```

```
        # веса обновляются после каждого примера
```

```
        for i in idx:
```

```
            Es += self.train_step(X[i], Y[i])
```

```
        # Проверка критерия остановки
```

```
        if Es <= Ee:
```

```
            return epoch, Es
```

```
    return max_epochs, Es
```

```
#
```

```
#=====
```

```
#
```

```
#
```

```
#
```

```
class SingleLayerPerceptron:
```

```
    def __init__(self, lr=0.5, seed=7):
```

```
        rng = np.random.RandomState(seed)
```

```
        # Два входа и один выход
```

```
        self.W = rng.uniform(-0.1, 0.1, (2, 1))
```

```
        # Порог
```

```
        self.T = rng.uniform(-0.1, 0.1, (1, 1))
```

```
        # Скорость обучения
```

```
        self.lr = lr
```

```
    def forward(self, x):
```

```
        """
```

```
        Прямое распространение.
```

```

"""

x = x.reshape(1, -1)

return sigmoid(x @ self.W - self.T)

def train_step(self, x, e):
    """
    Один шаг обучения.
    """

    x = x.reshape(1, -1)
    e = e.reshape(1, -1)

    y = self.forward(x)

    # Ошибка
    delta = (y - e) * dsigmoid(y)

    # Обновление весов
    self.W -= self.lr * (x.T @ delta)

    # Обновление порога
    self.T += self.lr * delta

    # Квадратичная ошибка
    error = 0.5 * np.sum((y - e) ** 2)

    return error

def predict(self, x):

    return self.forward(x)[0, 0]

def fit(self, X, Y, Ee=0.01, max_epochs=20000, seed=42):

    rng = np.random.RandomState(seed)

    for epoch in range(1, max_epochs + 1):

        idx = rng.permutation(len(X))

        Es = 0.0

        for i in idx:
            Es += self.train_step(X[i], Y[i])

        if Es <= Ee:
            return epoch, Es

```

```
return max_epochs, Es
```

```
# =====  
# ТОЧНОСТЬ МОДЕЛИ  
# =====
```

```
def accuracy(model, Xn, Y, thr=0.5):
```

```
    correct = 0
```

```
    for x, y in zip(Xn, Y):
```

```
        prediction = model.predict(x)
```

```
        predicted_class = int(prediction >= thr)
```

```
        correct_class = int(y[0])
```

```
        if predicted_class == correct_class:
```

```
            correct += 1
```

```
    return correct / len(Xn)
```

```
# =====  
# ВЫВОД ТАБЛИЦЫ РЕЗУЛЬТАТОВ  
# =====
```

```
def show_training_results(model, name):
```

```
    print()
```

```
    print("=" * 60)
```

```
    print(name)
```

```
    print("=" * 60)
```

```
    print(
```

```
        f'{'A':>8}'
```

```
        f'{'B':>8}'
```

```
        f'{'Ожидается':>15}'
```

```
        f'{'Выход':>15}'
```

```
        f'{'Класс':>12}'
```

```
    )
```

```
    print("-" * 60)
```

```
    for x_raw, x_norm, y_raw in zip(X, Xn, Y_raw):
```

```
        prediction_norm = model.predict(x_norm)
```

```
        prediction = denormalize(
```

```
            prediction_norm,
```

```

        c0,
        c1
    )

    predicted_class = (
        c0
        if prediction_norm < 0.5
        else c1
    )

    print(
        f'{x_raw[0]:>8.0f}'
        f'{x_raw[1]:>8.0f}'
        f'{y_raw[0]:>15.0f}'
        f'{prediction:>15.4f}'
        f'{predicted_class:>12}'
    )

# =====
# РЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ
# =====

def run_inference(model, A, B, c0, c1, name="Модель"):

    # Нормализация введённых значений
    x = normalize(
        np.array([A, B], dtype=float),
        c0,
        c1
    )

    # Предсказание
    y_norm = model.predict(x)

    # Возвращаем результат в исходный диапазон
    y_hat = denormalize(
        y_norm,
        c0,
        c1
    )

    # Определяем ближайший класс
    nearest = c0 if y_norm < 0.5 else c1

    print(
        f'[{name}] '
        f'A={A}, B={B} -> '
        f'ŷ={y_hat:.3f} -> '
        f'ближе к классу {nearest}'
    )

```

```
return y_hat, nearest
```

```
#=====
# ОСНОВНАЯ ПРОГРАММА
#=====
```

```
if __name__ == "__main__":
```

```
    print("=" * 60)
    print("ЛАБОРАТОРНАЯ РАБОТА №1")
    print("Решение задачи XOR")
    print("Вариант 3: c0 = -9, c1 = -8")
    print("=" * 60)
```

```
    print("\nТаблица истинности:")
```

```
    print("""
    A   B   A XOR B
-9   -9   -9
-9   -8   -8
-8   -9   -8
-8   -8   -9
    """)
```

```
#-----
# Обучение MLP
#-----
```

```
mlp = MLP221()
```

```
mlp_epochs, mlp_Es = mlp.fit(
    Xn,
    Y,
    Ee=0.01,
    max_epochs=20000
)
```

```
#-----
# Обучение однослойного персептрона
#-----
```

```
slp = SingleLayerPerceptron()
```

```
slp_epochs, slp_Es = slp.fit(
    Xn,
    Y,
    Ee=0.01,
    max_epochs=20000
)
```

```
# -----  
# Основные результаты  
# -----
```

```
mlp_acc = accuracy(  
    mlp,  
    Xn,  
    Y  
)
```

```
slp_acc = accuracy(  
    slp,  
    Xn,  
    Y  
)
```

```
print("\n" + "=" * 60)  
print("РЕЗУЛЬТАТЫ ОБУЧЕНИЯ")  
print("=" * 60)
```

```
print(  
    f"\nMLP 2-2-1:"  
    f"\nКоличество эпох: {mlp_epochs}"  
    f"\nСуммарная ошибка Es: {mlp_Es:.5f}"  
    f"\nAccuracy: {mlp_acc:.2f}"  
)
```

```
print(  
    f"\nОднослойный персептрон:"  
    f"\nКоличество эпох: {slp_epochs}"  
    f"\nСуммарная ошибка Es: {slp_Es:.5f}"  
    f"\nAccuracy: {slp_acc:.2f}"  
)
```

```
# -----  
# Таблица результатов MLP  
# -----
```

```
show_training_results(  
    mlp,  
    "MLP 2-2-1"  
)
```

```
# -----  
# Таблица результатов SLP  
# -----
```

```

show_training_results(
    slp,
    "ОДНОСЛОЙНЫЙ ПЕРСЕПТРОН"
)

# -----
# Ручной режим проверки
# -----

print()
print("=" * 60)
print("РЕЖИМ ФУНКЦИОНИРОВАНИЯ")
print("=" * 60)

print(
    "Введите два числа А и В "
    "из диапазона [-10; 10]."
)

print(
    "Например: -9 -8"
)

print(
    "Чтобы завершить программу, "
    "нажмите Enter."
)

while True:

    raw = input(
        "\nВведите А В "
        "(пусто — выход): "
    ).strip()

    if not raw:
        print("Программа завершена.")
        break

    try:

        A, B = map(
            float,
            raw.split()
        )

        if not (-10 <= A <= 10 and -10 <= B <= 10):

            print(
                "Ошибка: числа должны находиться "

```

```

        "в диапазоне [-10; 10]."
```

)

```

    continue

run_inference(
    mlp,
    A,
    B,
    c0,
    c1,
    "MLP"
)

run_inference(
    slp,
    A,
    B,
    c0,
    c1,
    "SLP"
)

except ValueError:

    print(
        "Ошибка ввода. "
        "Введите два числа через пробел."
    )
```

Результат тестирования:

ЛАБОРАТОРНАЯ РАБОТА №1

Решение задачи XOR

Вариант 3: c0 = -9, c1 = -8

Таблица истинности:

A	B	A XOR B
-9	-9	-9
-9	-8	-8
-8	-9	-8
-8	-8	-9

РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

MLP 2-2-1:

Количество эпох: 3062

Суммарная ошибка Es: 0.01000

Accuracy: 1.00

Однослойный персептрон:

Количество эпох: 20000

Суммарная ошибка Es: 0.53145

Accuracy: 0.75

MLP 2-2-1

A	B	Ожидается	Выход	Класс
-9	-9	-9	-8.9441	-9
-9	-8	-8	-8.0666	-8
-8	-9	-8	-8.0666	-8
-8	-8	-9	-8.9116	-9

ОДНОСЛОЙНЫЙ ПЕРСЕПТРОН

A	B	Ожидается	Выход	Класс
-9	-9	-9	-8.4922	-8
-9	-8	-8	-8.4962	-8
-8	-9	-8	-8.4995	-8
-8	-8	-9	-8.5035	-9

РЕЖИМ ФУНКЦИОНИРОВАНИЯ

Введите два числа A и B из диапазона [-10; 10].

Например: -9 -8

Чтобы завершить программу, нажмите Enter.

Введите A B (пусто — выход): -9 -9

[MLP] A=-9.0, B=-9.0 -> $\hat{y}$ =-8.944 -> ближе к классу -9

[SLP] A=-9.0, B=-9.0 -> $\hat{y}$ =-8.492 -> ближе к классу -8

Введите A B (пусто — выход): -9 -9

[MLP] A=-9.0, B=-9.0 -> $\hat{y}$ =-8.944 -> ближе к классу -9

[SLP] A=-9.0, B=-9.0 -> $\hat{y}$ =-8.492 -> ближе к классу -8

Введите A B (пусто — выход): -8 -9

[MLP] A=-8.0, B=-9.0 -> $\hat{y}$ =-8.067 -> ближе к классу -8

[SLP] A=-8.0, B=-9.0 -> $\hat{y}$ =-8.500 -> ближе к классу -8

Введите A B (пусто — выход): -5 -5

[MLP] A=-5.0, B=-5.0 -> $\hat{y}$ =-8.961 -> ближе к классу -9

[SLP] A=-5.0, B=-5.0 -> $\hat{y}$ =-8.538 -> ближе к классу -9

Введите A B (пусто — выход): 0 0

[MLP] $A=0.0, B=0.0 \rightarrow \hat{y}=-8.961 \rightarrow$ ближе к классу -9

[SLP] $A=0.0, B=0.0 \rightarrow \hat{y}=-8.593 \rightarrow$ ближе к классу -9

Вывод: Прodelав данную лабораторную работу я реализовал многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провел сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.